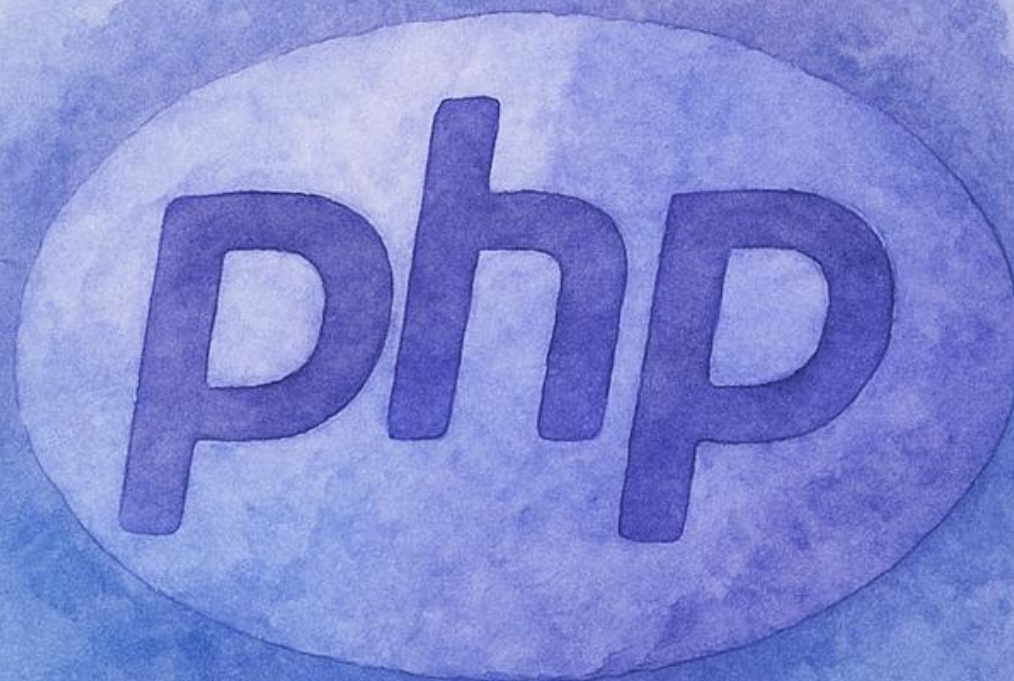


UD2 INTRODUCCIÓN A PHP

2º CFGS
Desarrollo de
Aplicaciones
Web



2025-2026

Alejandro Torres Zahinos



Importante de cara a todo el curso

Los nombres de variables, funciones, métodos, clases... deben escribirse en **inglés**.

Los nombres de los archivos también deben ser en **inglés**.

Se debe utilizar comentarios **PHPDoc al principio de cada archivo** al menos con las etiquetas **@author** y **@versión**.

También se deben utilizar comentarios **PHPDoc al principio de cada clase y de cada método**.

Evidentemente el **código** debe ser **auto explicativo** y en caso de haber elementos o algoritmos complejos en el código también se deben **comentar**.

1.- PHP

Lenguaje de programación de **propósito general**.

Su uso se ha extendido principalmente en el **desarrollo web**.

Su **sintaxis** es similar a **C** y **Java**.

Es un lenguaje **interpretado**.



1.- PHP

El mejor sitio para consultar la sintaxis y las funciones de PHP es en la documentación oficial <https://www.php.net/manual/es>.

La documentación oficial siempre se encuentra actualizada.

En la documentación oficial se indica las **versiones compatibles** de las funciones del lenguaje.



2.- Integración de PHP en HTML

Los archivos con código PHP tienen la extensión **.php** y habitualmente se les llama **scripts**.

Un **script PHP** (archivo .php) es **un archivo HTML** que además de etiquetas HTML puede contener código PHP que el servidor web ejecutará.

Los bloques de código PHP se escriben entre los delimitadores **<?php** y **?>**.

```
<?php
    echo "Desarrollo Web Entorno Servidor";
?>
```

Las expresiones PHP han de acabar con ;.

2.- Integración de PHP en HTML

Un **script PHP** dentro puede contener:

- Solo código HTML.
- Solo código PHP.
- Solo código PHP que además genera código HTML.
- Combinación de los tres puntos anteriores.

Práctica 1

Crea un proyecto en Laragon llamado **prueba**.

Crea el archivo **test.php** en la carpeta **prueba** (del proyecto recién creado).

Añade el siguiente código al archivo.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Primera prueba php</title>
</head>
<body>
  Este es un archivo php que se encuentra en el servidor.
</body>
</html>
```

Entra en el navegador y accede a la URL: <http://prueba.test/test.php>

Práctica 2

Crea el archivo **info.php** en la carpeta **prueba** (del proyecto creado anteriormente).

Añade el siguiente código al archivo.

```
<?php  
phpinfo();
```

Entra en el navegador y accede a la URL: <http://prueba.test/info.php>

Localiza: la versión de PHP y el directorio donde se encuentra el script ejecutado.

***Si un script PHP solo contiene código PHP no se debe poner el delimitador de cierre para evitar que se añadan espacios en blanco o líneas accidentales.**

Práctica 3

Crea el archivo **myname.php** en la carpeta **prueba** (del proyecto creado anteriormente).

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Presentación</title>
</head>
<body>
  <?php
    $name = "Álex Torres";
    echo 'Hola yo soy '. $name;
  ?>

  <br>
  Hola yo soy <?=$name?>
</body>
</html>
```

Entra en el navegador y accede a la URL: <http://prueba.test/myname.php>

Práctica 4

Crea el archivo **addition.php** en la carpeta **prueba** (del proyecto creado anteriormente).

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Suma</title>
</head>
<body>
  <?php
    $operando1 = 5;
    $operando2 = 26;

    echo 'La suma de los dos operandos es: '. $operando1+$operando2;
  ?>
</body>
</html>
```

Entra en el navegador y accede a la URL: <http://prueba.test/addition.php>

¿Qué ocurre internamente en el servidor cada vez que desde el navegador se accede a esa URL o se recarga la página?

2.- Integración de PHP en HTML

Tanto el servidor como los navegadores web disponen de memoria cache.

En ocasiones los cambios en las páginas web tardan en visualizarse en el navegador.

Para ayudar a que el navegador web cargue correctamente la última versión de la página web se recomienda añadir la siguiente línea en el **head**.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="expires" content="Sat, 07 feb 2016 00:00:00 GMT">
  <title>Título de la página</title>
</head>
```

2.- Integración de PHP en HTML

El código PHP incluido en el script se ejecuta en el servidor y habitualmente como se ha visto en las **Prácticas 3 y 4** puede generar contenido HTML.

Hay 3 maneras de generar contenido HTML desde PHP:

- **echo** *expresión*;
- **print** *expresión*;
- **<?= variable ?>** →



```
<?php
    echo $name;
?>
```

Ni **echo** ni **print** son funciones, por eso no es necesario poner paréntesis.

La última opción no se debe poner dentro de un bloque PHP.

En la **Práctica 3** se ha visto que mediante el operador `.` se pueden concatenar cadenas.

2.- Integración de PHP en HTML

Pasos en el servidor web al recibir una petición de un archivo .php:

El servidor busca el archivo (si no lo encuentra envía el error 404).

El servidor analiza el archivo y si existe código que debe importar lo hace. Obteniendo así un script php con todo el código necesario.

El servidor analiza el archivo buscando funciones/métodos/clases, si las encuentra, las carga en memoria y las elimina del archivo.

Con el script php definitivo, el servidor comienza a **interpretar** el archivo línea por línea. Si la línea es código HTML lo copia directamente al fichero de salida (código HTML), si encuentra un bloque PHP lo ejecuta pudiendo añadir más código HTML al fichero de salida.

Programming tip: Cadenas de texto

En las etiquetas del lenguaje HTML se suelen utilizar atributos, el valor de estos se indica entre **comillas dobles**:

```
<a href="photos.php">fotos</a>
```

Como en PHP es habitual generar código HTML, **para declarar y usar cadenas se recomienda el uso de comillas simples**.

```
<?php  
|     echo '<a href="photos.php">fotos</a>';  
?>
```

Si se quiere imprimir una comilla simple se tendrá que usar la secuencia de escape: \'

3.- Comentarios

Comentarios de una línea:

```
// Función para Loguearse
```

Comentarios de varias líneas:

```
/*  
    Script desarrollado por:  
    Álex Torres  
    Versión: 1.2  
*/
```

El código comentado en PHP no aparecerá en documento HTML final.

Recuerda que los comentarios en HTML sí que aparecerán: `<!--` `-->`

3.- Comentarios

Comentarios con phpDoc:

Es **recomendable** utilizar **comentarios phpDoc** de manera que el IDE los reconozca y que posteriormente se pueda **generar la documentación** de manera automática.

```
/**  
 * Bloque de comentario phpDoc  
 *  
 *  
 */
```

Son similares a los comentarios JavaDoc de Java.

3.- Comentarios

Comentarios con phpDoc:

Con phpDoc es recomendable comentar todos los elementos estructurales:

- Archivo
- Código importado: require e include
- Clases
- Interfaces
- Rasgos (traits)
- Funciones y métodos
- Propiedades de las clases
- Constantes

3.- Comentarios

Comentarios con phpDoc:

Antes de continuar echemos un vistazo a la [cómo documentar con phpDoc](#):

```
/**
 * This is the summary for a DocBlock.
 *
 * This is the description for a DocBlock. This text may contain
 * multiple lines and even some _markdown_.
 *
 * * Markdown style lists function too
 * * Just try this out once
 *
 * The section after the description contains the tags; which provide
 * structured meta-data concerning the given element.
 *
 * @author Mike van Riel <me@mikevanriel.com>
 *
 * @since 1.0
 *
 * @param int $example This is an example function/method parameter description.
 * @param string $example2 This is a second example.
 */
```

3.- Comentarios

Comentarios con phpDoc:

En la documentación se pueden ver las [etiquetas disponibles](#):

Tag reference

- [@api](#)
- [@author](#)
- [@category](#)
- [@copyright](#)
- [@deprecated](#)
- [@example](#)
- [@filesource](#)
- [@global](#)
- [@ignore](#)
- [Tag reference](#)

3.- Comentarios

Comentarios con phpDoc:

Para generar la documentación phpDoc se necesita incluir en el directorio del proyecto el [archivo **phpDocumentor.phar**](#).

A continuación, se debe ejecutar por terminal desde el directorio del proyecto el siguiente comando:

```
php phpDocumentor.phar -d . -t docs/api
```

De esta manera en el directorio del proyecto se creará la ruta **docs/api** y dentro se generará la documentación en formato HTML.

4.- Inclusión de ficheros externos

Al desarrollar un programa es habitual que se **reutilice el código**.

Esto puede ocurrir con **clases, librerías de funciones, ...** y en el desarrollo web también con secciones de la aplicación web.

En PHP existen las siguientes instrucciones para reutilizar archivos php:

```
include($_SERVER['DOCUMENT_ROOT'] . '/ruta_al_archivo_php');  
include_once($_SERVER['DOCUMENT_ROOT'] . '/ruta_al_archivo_php');  
  
require($_SERVER['DOCUMENT_ROOT'] . '/ruta_al_archivo_php');  
require_once($_SERVER['DOCUMENT_ROOT'] . '/ruta_al_archivo_php');
```

Más adelante se explicará qué es `$_SERVER['DOCUMENT_ROOT']`

4.- Inclusión de ficheros externos

Estas funciones en caso de encontrar el archivo añaden todo el contenido en el punto donde se encuentran.

Las funciones **include** si no encuentran el archivo muestran una advertencia y continúan la ejecución.

Las funciones **require** si no encuentran el archivo muestran un error y detienen la ejecución.

Las versiones **_once** aseguran que el código solo se añada **una vez**, esto permite evitar errores, por ejemplo, si el archivo contiene una función.

4.- Inclusión de ficheros externos

Es habitual que si un script php está pensado para que siempre se use se haga mediante su inclusión con **include** o **require** se le ponga la extensión **.inc.php**.

Por ejemplo:

header.inc.php

functions.inc.php

Para mantener organizados los archivos de la aplicación, los archivos php pensados para ser incluidos se almacenan en una carpeta llamada **includes** dentro del directorio raíz de la aplicación web.

4.- Inclusión de ficheros externos

Como **HTML no es un lenguaje de programación** no tiene ninguna herramienta que permita la inclusión de código HTML externo.

Por esta razón en las aplicaciones web estáticas es difícil reutilizar el código de manera sencilla.

En la realización de la actividad **DAW SHOP** para que todas las secciones tuvieran la misma cabecera, menú, pie y estilo se tuvo que **copiar el código en los diferentes archivos html**.

Gracias a las funciones **include** y **require** se soluciona el problema anterior.

Práctica

Actividad 1: DAW SHOP v2.0

5.- Tipos de datos

Los tipos de datos disponibles en PHP son:

Tipo	Descripción	Ejemplo
Boolean	Verdadero o falso	false
Integer	Número sin decimales	58
Float	Número con decimales	4.7
String	Conjunto de datos delimitados por comillas simples o dobles	'Rick Sanchez'
Array	Conjunto de datos del mismo o diferente tipo	'Rick', 'Sanchez', 70, 'humano'
Null	No hay valor asignado	null

5.- Tipos de datos

Boolean: valores posibles.

Verdadero	Falso
true	false
TRUE	FALSE
	0
	-0
Cualquier número entero o decimal, positivo o negativo distinto de cero.	0.0
	-0.0
	'0'
	"
Cualquier cadena que no sea cero o cadena vacía	Null

6.- Variables

Las variables permiten **almacenar datos** de algún tipo.

Los nombres de variables deben comenzar por **\$** seguido de **una letra** o el símbolo **_** y **no** se deben usar caracteres no ingleses y símbolos.

Algunos ejemplos válidos:

\$age	\$quantity_of_balls	\$_floor
\$_3types	\$first_surname	\$second_surname
\$street	\$i	

Los nombres son **case sensitive**, así, \$name es diferente a \$Name.

6.- Variables

PHP es un lenguaje de programación **débilmente tipado**, eso significa:

- Para empezar a usar una variable simplemente se le ha de asignar un valor.
- En cualquier momento se puede cambiar el tipo de dato que almacena una variable.

Se usa el operador = para asignar valor a una variable:

```
$variable = 5;  
$variable = 'Morty';  
$variable = true;
```

6.- Variables

Se pueden eliminar variables con la función **unset**.

```
$show = 'Rick & Morty';  
$seasons = 5;  
$name = 'Rick';  
$surname = 'Sanchez';  
  
unset($show);  
  
unset($seasons, $name, $surname);
```

7.- Variables y tipos de datos

Casting → Convertir de un tipo de dato a otro tipo de dato.

Type Juggling → si en una **operación** intervienen **variables de distinto tipo** PHP intentará convertirlas a un tipo común.

```
$quantity = 15;      // Valor entero
$price = 3.95;       // Valor decimal

// Type Juggling a float
$total = $quantity * $price;
echo 'El precio final es ' . $total . ' €<br>';

// Casting forzado a entero
$total = $quantity * (int)$price;
echo 'El precio final es ' . $total . ' €<br>';
```



```
El precio final es 59.25 €
El precio final es 45 €
```

8.- Constantes

Las constantes almacenan valores que no pueden variar durante la ejecución del programa.

Los nombres de las constantes son en **mayúsculas**.

Hay dos maneras de crear una constante:

```
define('PI', 3.141592);  
const NAME = 'Morty Smith'; // a partir de PHP 5.3
```



Existen [constantes predefinidas](#) que se pueden consultar en la documentación.

9.- Fechas y horas

Hasta la versión de 5.2 en PHP existía un tipo de datos concreto para fechas y horas.

Se utilizaba la marca de tiempo UNIX:

Segundos que han pasado desde las 00:00:00 del 1/1/1970

La función **time** devuelve el instante actual en formato UNIX.

```
$start = time();  
/* instrucciones PHP */  
echo 'Página generada en '. time()-$start .' segundos';
```

Aunque esta manera de trabajar con fechas y horas aún existe, desde PHP se recomienda el uso de la clase **DateTime**

9.- Fechas y horas

La clase **DateTime** es más potente, flexible y sencilla de utilizar.

Al instanciar un objeto con el constructor de [DateTime](#) se obtiene la fecha y hora actual.

Y → año con 4 cifras

m → mes con 2 cifras

d → día

H → hora en 24h

i → minutos

s → segundos

```
$now = new DateTime();  
echo $now->format('Y-m-d H:i:s');
```

En la [documentación](#) se pueden consultar todas las opciones de formato.

9.- Fechas y horas

Algunos ejemplos:

```
// Añadir un mes a la fecha
$date->modify('+1 month');

// Diferencia entre fechas
$date1 = new DateTime('2001-09-10');
$date2 = new DateTime('2025-12-25');
$interval = $date1->diff($date2);
echo $interval->format('%m meses y %d días') . '<br>';
if ($interval->format('%Y')>18) {
    echo 'Es mayor de edad<br>';
}
```

Se pueden usar los métodos **modify**, **setDate** y **setTime** para modificar el valor de un objeto DateTime.

9.- Fechas y horas

Si el servidor web se encuentra configurado en una zona horaria distinta a la zona horaria desde donde se visita la aplicación web será necesario cambiar en el script la zona para almacenar/mostrar correctamente las horas.

Al principio de los scripts en los que se trabaje con fechas y horas se deberá añadir la siguiente línea:

```
date_default_timezone_set('Europe/Madrid');
```

NOTA: en clase ocurre esto ya que el servidor web de Laragon no está configurado en la zona horaria de Madrid.

10.- Expresiones y operadores

Mediante los operadores se realizan operaciones con variables creando así expresiones.

Los operadores disponibles en PHP son similares a los operadores de Java.

Tipo de operación	Operadores disponibles										
Asignación	= += -= *= /= %=										
Aritmética	+ - * / % ** ++ --										
Comparación	>	<	>=	<=	==	!=	<>	!==	===	<=>	??
Comparación booleana	&& ! and or xor										
Sobre bits (integer)	& ^ (xor) ~(not) << >>										
Concatenación (strings)	. .=										

La prioridad en los operadores es **de izquierda a derecha** y siempre en este orden:
paréntesis, negación (!), producto/división, sumas/restas, comparación, lógicos y asignación

10.- Expresiones y operadores

A partir de PHP7

Nave espacial: $\$a \leq \b

Devuelve:

-1 si \$a es menor que \$b

0 si \$a es igual que \$b

1 si \$a es mayor que \$b

Fusión de null: $\$a ?? \$b ?? \$c$

Tantas variables como se quiera.

Devuelve el primer operando desde la izquierda que no sea null.

11.- Ámbito de las variables

Existen tres ámbitos posibles para una variable:

- **Local al script:** a partir de que se crea una variable, esta ya se puede usar en el resto del script PHP, aunque la variable se cree dentro de una estructura de control (if, for...). Pero no se podrá usar dentro de una función.
- **Local a una función:** si la variable se crea dentro de una función, esta solo se podrá usar dentro de la propia función.
- **Global:** una variable **local al script** se podrá usar dentro de una función si se indica con la palabra reservada **global**. **Se debe evitar este uso.**

11.- Ámbito de las variables

■ Local al script:

Si la variable se crea en el script o en una estructura de control, esa variable se podrá usar en todo el script excepto en las funciones.

```
<?php
    $i = 100;
    $temp = 0;
    while ($i!=0) {
        $temp += $i--;
        $addition = $temp;
    }
    echo 'variable temp: ' . $temp;
    echo '<br>';
    echo 'La suma de los 100 primeros números: ' . $addition;
```



11.- Ámbito de las variables

■ Local al script:

Si la variable se crea en el script o en una estructura de control, esa variable se podrá usar en todo el script excepto en las funciones.

```
<?php
$name = 'Álex';
greeting();

function greeting() {
    echo '¡Hola soy '. $name .'!'; //Error
}
```



11.- Ámbito de las variables

■ Local a una función:

Si la variable se crea dentro de una función, esa variable solo se podrá usar en esa función.

```
<?php
greeting();
echo $name; // Error ←

function greeting() {
    $name = 'Alex'; ←
    echo '¡Hola soy ' . $name . '!';
}
```

11.- Ámbito de las variables

■ Global:

Si la variable se crea en el script fuera de una función, esa variable se podrá usar en las funciones del script si se usa la palabra reservada global.

```
<?php
$name = 'Álex'; ←
greeting();

function greeting() {
    global $name; ←
    echo '¡Hola soy '. $name . '!';
}
```

Se debe evitar siempre el uso de variables globales.

12.- Estructuras de control

PHP dispone de sentencias que permiten alterar el flujo de ejecución.

Estas sentencias son similares a las de Java:

if switch while do-while for

12.- Estructuras de control

if y if – else

```
if ($number == 0) {  
    echo 'El número es cero.';  
}  
  
if ($a < $b) {  
    echo 'a es menor que b';  
} else if ($a > $b) {  
    echo 'a es mayor que b';  
} else {  
    echo 'a es igual a b';  
}
```

```
if ($number == 0)  
    echo 'El número es cero.';  
  
if ($a < $b)  
    echo 'a es menor que b';  
else if ($a > $b)  
    echo 'a es mayor que b';  
else  
    echo 'a es igual a b';
```

12.- Estructuras de control

Operador ternario

Es una manera abreviada de la estructura if – else.

condición ? si_verdadero : si_falso

```
$price = 199.5;  
$discount = 15;  
$finalprice = ($discount > 0) ? $price-$price*$discount/100 : $price;
```

12.- Estructuras de control

Operador ternario

Aporta mucha versatilidad.

```
echo 'Bienvenido ';  
if (isset($username))  
|   echo $username;  
else  
|   echo 'Invitado';  
echo '!!';
```

```
echo 'Bienvenido '. (isset($username)?$username:'Invitado') . '!!';
```

Este ejemplo se puede mejorar aún más con el operador fusión de null.

```
echo 'Bienvenido '. ($username??'Invitado') . '!!';
```

12.- Estructuras de control

switch

```
$numero = rand(0,1); // entero aleatorio entre 0 y 1

switch ($numero) {
    case 0: echo 'numero vale 0';
            break;
    case 1: echo 'numero vale 1';
            break;
    default: echo 'numero no vale ni 0 ni 1';
}
```

```
$numero = rand(0,1); // entero aleatorio entre 0 y 1

switch ($numero) {
    case 0: echo 'numero vale 0';
    case 1: echo 'numero vale 1';
            break;
    default: echo 'numero no vale ni 0 ni 1';
}
```

¿Qué hace este código?

12.- Estructuras de control

while

```
$numero = 1;
while ($numero < 8) {
    $numero += 3;
}
echo $numero;
```

do – while

```
$numero = 5;
do {
    $numero -= 3;
} while ($numero > 10);
echo $numero;
```

¿Qué hace este código?

12.- Estructuras de control

for

```
for ($i=0; $i < 10; $i++) {  
    echo $i;  
    echo '<br>';  
}
```

```
for ($i=5; $i < 10; $i+=3) {  
    echo $i;  
    echo '<br>';  
}
```

¿Qué hace este código?

Práctica

Actividad 2:

Generando HTML

Actividad 3:

Calculando

Actividad 4:

Las tablas de multiplicar

13.- Arrays

En PHP los **arrays** son **mapas ordenados** estos se pueden emplear como **array, lista, tabla hash, diccionario, pila, cola...**

Los mapas no tienen un tamaño establecido pudiendo añadir y eliminar elementos en cualquier momento.

Los mapas permiten almacenar pares **clave – valor**.

Las claves son los índices para acceder a los elementos del array

13.- Arrays

Los arrays pueden ser de 3 tipos:

- **Numérico:** todos los índices son integer (números enteros).
- **Asociativo:** todos los índices son strings.
- **Mixto:** tiene índices integer y strings.

13.- Arrays

Los arrays se pueden definir de dos maneras diferentes:

```
$suits = array('Corazones', 'Picas', 'Rombos', 'Tréboles');  
  
$characters = ['Rick', 'Morty', 'Summer', 'Beth', 'Jerry'];
```

Si no se indica la clave, automáticamente se asignará un **valor numérico** comenzando desde **cero** → **array numérico**.

\$suits[0] almacena la cadena: Corazones

\$characters[3] almacena la cadena: Beth

13.- Arrays

Se puede añadir elementos al final de un array de la siguiente forma:

```
$characters = ['Luke', 'Leia'];  
$characters[] = 'Han Solo';      //Se asigna la clave 2
```

Mediante la clave (índice) también se puede añadir elementos a un array:

```
$characters[3] = 'Darth Vader';
```

Se debe tener cuidado con esta práctica si se indica una clave mayor que la última posición del array:

```
$characters[6] = 'R2D2'; // Las claves 4 y 5 no existen  
$characters[] = 'Yoda';  // se asigna la clave 7
```

13.- Arrays

De esta manera también se puede inicializar un array:

```
unset($colors);  
$colors[] = 'azul'; // Crea el array y añade un elemento
```

Si la variable contenía anteriormente un tipo de dato para poder crear un array en ella primero se debe eliminar la variable:

```
$show = 'Stranger Things';  
unset($show);  
$show[] = 'Breaking bad';  
$show[] = 'Black Mirror';
```

13.- Arrays

Un mismo array en PHP puede almacenar varios tipos de datos a la vez.

```
$character[] = 'Rick Sanchez'; // nombre
$character[] = 70;           // edad
$character[] = 1.98;         // altura en cm
$character[] = 86;           // peso en Kg
$character[] = 'humano';     // especie
$character[] = 'C-137';      // dimensión
$character[] = true;         // vivo
```

13.- Arrays

En los **arrays asociativos** todas las claves son strings.

Se pueden definir con todas las técnicas vistas anteriormente pero **añadiendo la clave:**

```
$character = array('name' => 'Rick Sanchez',  
                  'age' => 70,  
                  'height' => 1.98);
```

```
$character = ['name' => 'Rick Sanchez',  
             'age' => 70,  
             'height' => 1.98];
```

```
$character['name'] = 'Rick Sanchez';  
$character['age'] = 70;  
$character['height'] = 1.98;
```

13.- Arrays

Al indicar una clave, PHP intenta hacer un **casting a integer**, por eso hay que tener cuidado con las claves

```
$array = [1 => 'a',  
          '1' => 'b',  
          1.5 => 'c',  
          true => 'd'];
```

Al hacer casting a integer todas las claves anteriores tienen el valor 1 por lo que el array declarado solo contiene un elemento con la clave 1:

`$array[1]` contiene la cadena "d";

13.- Arrays

En los **arrays mixtos** se pueden mezclar claves integer y string.

```
$character['name'] = 'Rick Sanchez';  
$character[] = 70; // Se asigna la clave 0  
$character['height'] = 1.98;  
$character[] = 86; // Se asigna la clave 1  
$character[5] = 'humano';  
$character[] = 'C-137'; // Se asigna la clave 6  
$character[] = true; // Se asigna la clave 7
```

Como se puede observar, no tiene mucho sentido crear un array de esta manera debido a la confusión de trabajar con él.

13.- Arrays

Un uso típico de los arrays mixtos es poder **acceder a todos** sus elementos tanto con la clave integer como con la clave string.

```
$character = [0 => 'Rick Sanchez',  
             'name' => 'Rick Sanchez',  
             1 => 70,  
             'edad' => 70,  
             2 => 1.98,  
             'height' => 1.98,  
             3 => 'humano',  
             'species' => 'humano'];
```

```
$character[0] = 'Rick Sanchez';  
$character['name'] = 'Rick Sanchez';  
$character[1] = 70;  
$character['edad'] = 70;  
$character[2] = 1.98;  
$character['height'] = 1.98;  
$character[3] = 'humano';  
$character['species'] = 'humano';
```

Las claves integer no es necesario ponerlas porque como ya se ha visto, si no se indica clave, esta será integer empezando desde cero.

Algunas funciones de acceso a bases de datos devuelven arrays mixtos.

13.- Arrays

Se pueden **eliminar** elementos del array con la función unset.

```
unset($array[2]);  
  
unset($character['name']);
```

Si es un array numérico habrá que volver a crear los índices mediante la función **array_values**, para evitar problemas si se recorre todo el array.

```
$array = array_values($array);
```

13.- Arrays

Como en PHP un elemento de un array puede ser de cualquier tipo, para crear **matrices** o **arrays multidimensionales** simplemente habrá que indicar que el elemento del array es un array.

```
$numbers = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];

$films = [['title' => 'Matrix', 'year' => 1999],
           ['title' => 'Vengadores: Infinity War', 'year' => 2018],
           ['title' => 'Vengadores: Endgame', 'year' => 2019],
           ['title' => 'Rogue One', 'year' => 2016]];

$array = [['Ana', 'Luis', 'Marta'],
           ['colors' => ['amarillo', 'verde'],
            ['greeting' => 'hola', 'goodbye' => 'adiós', '¿Qué tal estás?']]];
```

13.- Arrays

Mediante la función **count** se obtiene la cantidad de elementos de un array.

Esta función permite recorrer un **array numérico** sin problema.

```
<ul>
    <?php
        $suits = ['Corazones', 'Picas', 'Rombos', 'Tréboles'];
        for ($i=0; $i<count($suits); $i++)
            echo '<li>'. $suits[$i] .'
```

- Corazones
- Picas
- Rombos
- Tréboles

13.- Arrays

Para recorrer un array numérico o asociativo es mejor usar **foreach**.

```
<ul>
    <?php
        $suits = ['Corazones', 'Picas', 'Rombos', 'Tréboles'];
        foreach ($suits as $value) {
            echo '<li>'. $value .'</li>';
        }
    ?>
</ul>
```

- Corazones
- Picas
- Rombos
- Tréboles

```
<?php
    $suits = ['Corazones', 'Picas', 'Rombos', 'Tréboles'];
    foreach ($suits as $key => $value) {
        echo $key .'.- ' . $value .'<br>';
    }
?>
```

- 0.- Corazones
- 1.- Picas
- 2.- Rombos
- 3.- Tréboles

13.- Arrays

Si se quiere modificar los elementos de un array desde un foreach se necesitan las posiciones del array.

```
$numbers = [1, 32, 6, 8, 45, 23, 14];  
foreach ($numbers as $number) {  
    // solo se modifica la variable temporal $number  
    $number++;  
}
```

```
$numbers = [1, 32, 6, 8, 45, 23, 14];  
foreach ($numbers as $key => $number) {  
    $numbers[$key]++;  
}
```

Práctica

Actividad 5: Ordenando datos

14.- Funciones predefinidas

PHP dispone de infinidad de funciones predefinidas ([referencia funciones](#)).

Es importante consultar la documentación para ver cómo **funciona** una función y cómo se realiza la **llamada** a la función.

empty

(PHP 4, PHP 5, PHP 7, PHP 8)

empty — Determina si una variable está vacía

Descripción

```
empty(mixed $var): bool
```

Determina si una variable es considerada vacía. Una variable se considera vacía si no existe o si su valor es igual a **false**. **empty()** no genera una advertencia si la variable no existe.

14.- Funciones predefinidas

Funciones para variables

Función	Descripción
empty	Comprueba si la variable está vacía (valor no vacío distinto de cero).
isset	Comprueba si una (o más variables) existe y no es null.
unset	Destruye una (o más variables).
is_int	Comprueba si una variable es integer.
is_float	Comprueba si una variable es float.
is_string	Comprueba si una variable es string.
is_array	Comprueba si una variable es un array
is_object	Comprueba si una variable es un objeto.
is_null	Comprueba si una variable es null.

14.- Funciones predefinidas

Funciones matemáticas

Función	Descripción
pow	Calcula la potencia (un número elevado a otro).
sqrt	Calcula la raíz cuadrada de un número.
round	Redondea un decimal dependiendo del dígito: 0 a 4 si hacia abajo, 5 a 9 hacia arriba.
ceil	Redondea un decimal hacia arriba.
floor	Redondea un decimal hacia abajo.
decbin	Convierte un número decimal a binario en forma de string.
bindec	Convierte un número binario en forma de string a decimal.
max	Encuentra el valor más alto entre las variables indicadas (también con array de integer).
min	Encuentra el valor más bajo entre las variables indicadas (también con array de integer).
sin, cos, tan...	Funciones trigonométricas.
rand	Genera un número aleatorio.
mt_rand	Genera un número aleatorio de una mejor manera que la anterior.

14.- Funciones predefinidas

Funciones para strings

Función	Descripción
strlen	Obtiene la longitud de una cadena.
substr	Devuelve una parte de una cadena.
str_replace	Sustituye parte de una cadena por otra cadena indicada.
strtolower	Transforma una cadena a minúsculas.
strtoupper	Transforma una cadena a mayúsculas.
trim	Elimina espacios al principio y al final.
explode	Divide un string en varios strings almacenándolos en un array.
implode	Une elementos de un array en un string.
strip_tags	Elimina etiquetas HTML.
strcmp	Comparación segura de dos cadenas.
strcasecmp	Comparación segura de dos cadenas convirtiéndolas primero a minúsculas.

14.- Funciones predefinidas

Funciones para arrays

Función	Descripción
count	Cuenta los elementos de un array
array_values	Devuelve un array con los valores del mismo.
array_keys	Devuelve un array con las claves del mismo.
array_key_exists	Comprueba si una clave existe en un array.
array_flip	Intercambia las claves y los valores de un array. Los valores pasarán a ser las claves y viceversa.
array_pop	Extrae el último elemento de un array.
array_push	Añade uno o más elementos al final de un array.
array_shift	Extrae el primer elemento del array. Si es numérico recalcula las claves para que empiecen en cero.
array_unshift	Añade uno o más elementos al inicio de un array.
array_reverse	Devuelve un array con los elementos en orden inverso al array indicado.
array_merge	Combina dos o más arrays.
in_array	Comprueba si un valor existe en un array.
sort	Ordena un array.

15.- Funciones propias

Para definir funciones propias se utiliza la palabra reservada **function**:

```
<?php
    function greeting () {
        echo 'Hola ¿Qué tal va?';
    }
?>
```

Para usar una función, esta debe estar definida en el mismo script desde donde se realiza la llamada, ya sea **directamente** o a través de **include** o **require**.

```
<?php
    greeting();
?>
```

15.- Funciones propias

Como se ha visto hasta el momento PHP es muy flexible.

Por esto a la hora de definir y usar funciones propias existen una serie de características que difieren de otros lenguajes de programación.

- Paso de argumentos por valor.
- Paso de argumentos por referencia.
- **Argumentos con valor por defecto/opcionales.**
- **Argumentos con nombre.**
- **Cantidad de argumentos.**
- Devolver un valor.
- Indicando el tipo de dato.

15.- Funciones propias

Paso de argumentos por valor.

Al definir una función se pueden indicar los argumentos que necesitará.

Lo más habitual es realizar el paso por **valor**, lo cual significa que se **realizará una copia** de la variable para usarla dentro de la función.

```
function addition($number1, $number2) {  
    echo $number1 . '+' . $number2 . '=' . $number1+$number2;  
}  
  
addition(14, 27);
```

15.- Funciones propias

Paso de argumentos por referencia.

Al pasar parámetros por **referencia no se crea una copia** de la variable y si se modifica dentro de la función los cambios se mantendrán fuera de esta.

```
$number = 6;  
function increment(&$number, $quantity) {  
    $number += $quantity;  
}  
  
echo 'Número antes de la llamada: ' . $number . '<br>';  
increment($number, 25);  
echo 'Número después de la llamada: ' . $number . '<br>';
```

15.- Funciones propias

Argumentos con valor por defecto/opcionales.

Se puede indicar un **valor por defecto** a los argumentos, estos argumentos además serán **opcionales**. Los argumentos con valor por defecto deben ser los últimos.

```
function final_price($price, $vat=21, $discount=0) {  
    echo 'Precio sin iva: ' . $price . '€<br>';  
    if($discount!=0) {  
        echo 'Descuento (' . $discount . '%): ' . $price*$discount/100 . '€<br>';  
        $price -= $price*$discount/100;  
    }  
    echo 'IVA (' . $vat . '%): ' . $price*$vat/100 . '€<br>';  
    echo 'Precio final: ' . $price + $price*$vat/100 . '€<br>';  
}  
  
final_price(150);  
final_price(150, 10);  
final_price(150, 10, 50);
```

15.- Funciones propias

Argumentos con valor por defecto/opcionales.

```
function leading_role($film="todas") {  
    $data = ['El señor de los anillos' => 'Frodo',  
            'Matrix' => 'Neo',  
            'Piratas del caribe' => 'Jack Sparrow',  
            'Iron man' => 'Tony Stark',  
    ];  
  
    if($film=="todas") {  
        foreach($data as $key => $value) {  
            echo $key .': '. $value .'.<br>';  
        }  
    }  
    else if(!array_key_exists($film, $data)) {  
        echo 'No hay datos de esa película.<br>';  
    }  
    else {  
        echo $film .': '. $data[$film] .'.<br>';  
    }  
}  
  
leading_role();  
leading_role('Matrix');  
leading_role('Harry Potter');
```

15.- Funciones propias

Argumentos con nombre.

Desde PHP 8 se pueden realizar llamadas a funciones con el **nombre del argumento**:

```
function final_price($price, $vat=21, $discount=0) { ...  
}  
  
final_price(price:150, discount:15);  
final_price(93, discount:7, vat:10);  
final_price(discount:15, price:199, vat:21);
```

Si se usan argumentos con y sin nombre en la misma llamada, los argumentos con nombre deben ser los últimos.

15.- Funciones propias

Cantidad de argumentos.

Aunque en la definición de la función se indiquen unos argumentos PHP al realizar la llamada a la función **se pueden indicar más argumentos** de los definidos, **nunca menos**.

```
function test_function() { ...  
}  
test_function();  
test_function(1, 2, 3, 4);  
  
function addition($number1, $number2) { ...  
}  
addition(); // Error  
addition(14, 27);  
addition(14, 27, 52);
```

15.- Funciones propias

Cantidad de argumentos.

Aunque se indiquen más argumentos, si en la definición de la función no se realiza ninguna acción con ellos no habrá ningún error.

```
function addition($number1, $number2) {  
    echo $number1 . '+' . $number2 . '=' . $number1+$number2;  
}  
addition(14, 27);  
addition(14, 27, 52, 9, 14);
```

Existen dos maneras de acceder a los argumentos no indicados en la definición:

- Mediante las funciones: `func_num_args` y `func_get_arg`.
- Creando un array (PHP > 5.6).

15.- Funciones propias

Cantidad de argumentos.

Mediante las funciones: `func_num_args` y `func_get_arg`.

```
function addition() {  
    $addition = 0;  
    for ($i=0; $i<func_num_args(); $i++) {  
        $addition += func_get_arg($i);  
    }  
    echo 'La suma de los números es: '. $addition;  
}  
  
addition(14, 27);  
addition(14, 27, 52, 9, 14);
```

15.- Funciones propias

Cantidad de argumentos.

Creando un array (PHP > 5.6).

```
function addition(...$numbers) {  
    $addition = 0;  
    foreach ($numbers as $number) {  
        $addition += $number;  
    }  
    echo 'La suma de los números es: '. $addition;  
}  
  
addition(14, 27);  
addition(14, 27, 52, 9, 14);
```

15.- Funciones propias

Cantidad de argumentos.

Mediante la notación de tres puntos ... se pueden definir parámetros concretos primero y al final dejar la opción de añadir más parámetros en la llamada.

```
function operation($operation, ...$numbers) {  
    $result = 0;  
    if ($operation === 'suma') {  
        foreach ($numbers as $number) {  
            $result += $number;  
        }  
    } else if ($operation === 'resta') {  
        foreach ($numbers as $number) {  
            $result -= $number;  
        }  
    } else {  
        $result = 'Operación no soportada.';  
    }  
    return $result;  
}  
  
echo operation('suma', 1, 6, 15, 5);
```

15.- Funciones propias

Cantidad de argumentos.

Los tres puntos ... también permiten separar un array en diferentes variables cuando se pasa el array como argumento a una función.

```
function final_price($prctice, $vat=21, $discount=0) { ...  
}  
  
$ps5 = [499.99, 21.0, 5.0];  
final_price(...$ps5);  
$switch = [299.5, 21.0];  
final_price(...$switch);
```

```
function addition(...$numbers) { ...  
}  
  
$numbers = [1, 2, 3, 4, 5];  
addition(...$numbers);
```

15.- Funciones propias

Devolver valores.

Mediante **return** una función puede devolver un dato al finalizar su llamada.

```
function addition(...$numbers) {  
    $result = 0;  
    foreach ($numbers as $number) {  
        $result += $number;  
    }  
    return $result;  
}
```

15.- Funciones propias

Indicando el tipo de dato.

Desde PHP 7 se permite indicar el tipo de dato de los argumentos y del valor devuelto. Si se definen las funciones así PHP se vuelve más estricto. Aunque no es necesario, se recomienda indicar el tipo de dato.

```
function final_price(int $price, float $vat=21, int $discount=0): float {  
    $price -= $price*$discount/100;  
    $price += $price*$vat/100;  
    return $price;  
}
```

Los tipos de datos son: int, float, string, array, object (o una clase concreta), bool y null.

Si un parámetro de entrada o de salida puede ser de cualquier tipo de dato se debe usar la palabra reservada **mixed**.

15.- Funciones propias

Cabe recordar que cuando PHP usa una variable siempre va a intentar hacer un casting con ella para evitar posibles problemas.

El siguiente ejemplo no produce error ya que al hacer casting no hay error.

```
function increment(...$numbers) {  
    $result = 0;  
    foreach($numbers as $number) {  
        $result += $number;  
    }  
    return $result;  
}  
  
echo increment(14, '2', true);
```

16.- PHP Standards Recommendations

Como todo lenguaje de programación, en PHP existen una serie de convenciones a la hora de escribir código.

Estas convenciones se recogen en los [PSR](#).

PSR (PHP Standards Recommendations).

Las PSR son recomendaciones que se pueden seguir o no.

Si se siguen el código será más estándar.

ACCEPTED	
NUM	TITLE
1	Basic Coding Standard
3	Logger Interface
4	Autoloading Standard
6	Caching Interface
7	HTTP Message Interface
11	Container Interface
12	Extended Coding Style Guide
13	Hypermedia Links
14	Event Dispatcher
15	HTTP Handlers
16	Simple Cache
17	HTTP Factories
18	HTTP Client

16.- PHP Standards Recommendations

Algunos ejemplos:

PSR-1: El nombre de los métodos debe ser lower camelCase.

PSR-12: Si el script PHP solo contiene código PHP se omite el `?>` final.

Se recomiendan 120 caracteres máximos por línea.

Se deben emplear 4 espacios para tabular (VSCode lo hace automáticamente).

Solo una instrucción por línea.

Las palabras reservadas deben escribirse en minúscula.

Debe haber un espacio tras las palabras if, else, switch, while, do, for.

No debe haber espacio tras el paréntesis de apertura ni antes del de cierre.

Debe haber un espacio entre el paréntesis de cierre y la llave de apertura.

El cuerpo de la estructura de control tiene que tener un tabulado.

Las palabras else y else if deben ir en la misma línea que la llave de cierre anterior.

Práctica

Actividad 6:
Calculando v2.0

Actividad 7:
Juegos de cartas